Homework 2:

Python Practice and Statistics & Probability

CPE 486/586

Instructor: Rahul Bhadani

Due: Sept 26, 2026, 11:59 PM
135 points (586), 115 Points (486)

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

Submission instruction:

You may either complete your entire homework on Python Notebook and submit a Google Colab link or you may choose a combination of a pdf and Google Colab Notebook. However, you must provide **publicly accessible** Google Colab URL on Canvas.

For .pdf on Canvas, follow the format `CPE486586-LastFirst-HW-XX`. For example, if your name is Sam Wells, your file name should be `CPE486586-WellsSam-HW-XX.pdf`.

Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

Please write down your unique keyword corresponding to your package and your Github account URL.

Theory

1 Disjoint Sets (10 points)

If $P(A) = \frac{1}{3}$ and $P(B^c) = \frac{1}{4}$, can A and B be disjoint? Explain.

Answer

If A and B were disjoint, $P(A \cup B) = P(A) + P(B) = 1/3 + 3/4 = 13/12$, which is impossible. More generally, if A and B were disjoint, then $A \subseteq B^c$ and $P(A) \leq P(B^c)$. But here $P(A) > P(B^c)$, so A and B cannot be disjoint.

2 Moment Generating Function (10 Points)

The moment generating function (MGF) of a random variable X, denoted as $M_X(t)$ is given by

$$M_X(t) = \mathbb{E}(e^{tX}) = \int_{-\infty}^{\infty} e^{tx} f_X(x) dx \tag{1}$$

for a continuous random variable.

Compute the MGF of a standard normal distribution with mean 0 and standard deviation 1.

Answer

PDF of a normal distribution:

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}} \tag{2}$$

Thus moment generating function is given by

$$M_X(t) = \mathbb{E}(e^{tX}) = \int_{-\infty}^{\infty} e^{tx} \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}} \tag{3}$$

Substituting $u = (x-\mu)/(\sqrt{2}\sigma)$, i.e. $x = \sqrt{2}\sigma u + \mu$, we have

$$\begin{aligned}
M_X(t) &= \frac{1}{\sqrt{2\pi}\sigma} \int_{(-\infty-\mu)/(\sqrt{2}\sigma)}^{(+\infty-\mu)/(\sqrt{2}\sigma)} \exp\left[t(\sqrt{2}\sigma u + \mu) - \frac{1}{2}\left(\frac{\sqrt{2}\sigma u + \mu - \mu}{\sigma} \right)^2 \right] d(\sqrt{2}\sigma u + \mu) \\
&= \frac{\sqrt{2}\sigma}{\sqrt{2\pi}\sigma} \int_{-\infty}^{+\infty} \exp\left[t(\sqrt{2}\sigma u + \mu) - u^2 \right] du \\
&= \frac{1}{\sqrt{\pi}} \int_{-\infty}^{+\infty} \exp[\sqrt{2}\sigma u t + \mu t - u^2] du \\
&= \frac{\exp(\mu t)}{\sqrt{\pi}} \int_{-\infty}^{+\infty} \exp[-(u^2 - \sqrt{2}\sigma u t)] du \\
&= \frac{\exp(\mu t)}{\sqrt{\pi}} \int_{-\infty}^{+\infty} \exp\left[-\left(u - \frac{\sqrt{2}}{2}\sigma t \right)^2 + \frac{1}{2}\sigma^2 t^2 \right] du \\
&= \frac{\exp\left[\mu t + \frac{1}{2}\sigma^2 t^2\right]}{\sqrt{\pi}} \int_{-\infty}^{+\infty} \exp\left[-\left(u - \frac{\sqrt{2}}{2}\sigma t \right)^2 \right] du
\end{aligned} \tag{4}$$

Now substituting $v = u - \sqrt{2}/2\,\sigma t$, i.e. $u = v + \sqrt{2}/2\,\sigma t$, we have

$$\begin{aligned}
M_X(t) &= \frac{\exp\left[\mu t + \frac{1}{2}\sigma^2 t^2\right]}{\sqrt{\pi}} \int_{-\infty-\sqrt{2}/2\,\sigma t}^{+\infty-\sqrt{2}/2\,\sigma t} \exp[-v^2]\, d(v + \sqrt{2}/2\,\sigma t) \\
&= \frac{\exp\left[\mu t + \frac{1}{2}\sigma^2 t^2\right]}{\sqrt{\pi}} \int_{-\infty}^{+\infty} \exp[-v^2]\, dv
\end{aligned} \tag{5}$$

With the Gaussian integral

$$\int_{-\infty}^{+\infty} \exp[-x^2]\, dx = \sqrt{\pi}$$

this finally becomes

$$M_X(t) = \exp\left[\mu t + \frac{1}{2}\sigma^2 t^2\right]$$

With $\mu = 0$, and $\sigma = 1$,

$$M_X(t) = \exp\left[\frac{1}{2}t^2\right] \quad \text{or} \quad e^{\frac{t^2}{2}} \tag{6}$$

3 Probability Density Function (10 Points)

Let X have the pdf

$$f(x) = \frac{4}{\beta^3\sqrt{\pi}} x^2 e^{-x^2} e^{-x^2/\beta^2}, \quad 0 < x < \infty, \quad \beta > 0 \tag{7}$$

Check whether $f(x)$ is a pdf.

Answer

Simplifying the exponential terms:

$$e^{-x^2} e^{-x^2/\beta^2} = e^{-x^2(1+1/\beta^2)} = e^{-x^2(\beta^2+1)/\beta^2} \tag{8}$$

our integral becomes:

$$\frac{4}{\beta^3\sqrt{\pi}} \int_0^\infty x^2 e^{-x^2(\beta^2+1)/\beta^2} dx$$

Let $u^2 = x^2 \cdot \frac{\beta^2+1}{\beta^2}$ Then:

$$x^2 = u^2 \cdot \frac{\beta^2}{\beta^2+1} \tag{9}$$

$$x = u \cdot \frac{\beta}{\sqrt{\beta^2+1}} \tag{10}$$

$$dx = \frac{\beta}{\sqrt{\beta^2+1}} du \tag{11}$$

The integral transforms to:

$$\begin{aligned} &\frac{4}{\beta^3\sqrt{\pi}} \int_0^\infty u^2 \cdot \frac{\beta^2}{\beta^2+1} \cdot e^{-u^2} \cdot \frac{\beta}{\sqrt{\beta^2+1}} du \\ &= \frac{4}{\beta^3\sqrt{\pi}} \cdot \frac{\beta^3}{(\beta^2+1)^{3/2}} \int_0^\infty u^2 e^{-u^2} du \\ &= \frac{4}{\sqrt{\pi}} \cdot \frac{1}{(\beta^2+1)^{3/2}} \int_0^\infty u^2 e^{-u^2} du \end{aligned} \tag{12}$$

Using the Gamma function:

$$\int_0^\infty u^2 e^{-u^2} du = \frac{1}{2}\Gamma\left(\frac{3}{2}\right) = \frac{1}{2} \cdot \frac{\sqrt{\pi}}{2} = \frac{\sqrt{\pi}}{4} \tag{13}$$

Therefore:

$$\frac{4}{\sqrt{\pi}} \cdot \frac{1}{(\beta^2+1)^{3/2}} \cdot \frac{\sqrt{\pi}}{4} = \frac{1}{(\beta^2+1)^{3/2}} \tag{14}$$

Clearly, PDF doesn't integrate to 1, thus the given $f(x)$ is not a valid PDF.

Practice

4 Dice Roll (10 points)

Write a Python program to simulate the rolling of a fair dice. After 100 tosses what is the probability of getting a 6? What about after 1000 rolls? What about after 10000 rolls?

5 Three coins toss and CDF (10 points)

Consider the experiment of tossing three fair coins, and let X = number of heads observed. The CDF of X is

$$F_X(x) = \begin{cases} 0, & -\infty < x < 0 \\ \frac{1}{8}, & 0 \leq x < 1 \\ \frac{1}{2}, & 1 \leq x < 2 \\ \frac{7}{8}, & 2 \leq x < 3 \\ 1, & 3 \leq x < \infty \end{cases} \tag{15}$$

Write a Python program to draw a graph $F_X(x)$ using step functions. Determine $F_X(2.5)$ using Python.

6 Probability Density Functions: Normal Distribution (10 points)

Refer to the Python Notebook https://github.com/rahulbhadani/CPE486586_FA26/blob/master/Code/CPE486586_Ch04_SyntheticDataGeneration.ipynb some related examples.

Write a Python program to generate 10000 random samples following a Normal distribution. Keep $\mu = 0.5$ fixed and vary the standard deviation $\sigma = 0.3$, 5.0, and 10.0. Plot the probability density function using histogram and kernel density estimation (KDE) plot. Comment on your observation about changing the standard deviation and its effect on the shape of the probability density function.

7 Data Exploration with Motorway Control System (MCS) Data (25 Points)

Download the MCS data from https://github.com/rahulbhadani/Datasets/blob/master/mcsOct2018.csv.

For this portion of the assignment, you will use **polars** Python package that you can install using `uv add polars` in your UV environment or `!pip install polars` in your Google Colab notebook.

Perform the following task:

1. Read the dataset into a polars data frame. How many columns the dataframe has? **(2 Points)**
2. Convert date column to POSIX format, sort values chronologically. POSIX time represents the number of seconds since January 1, 1970 00:00:00 UTC. This is a common way to represent time in Linux and is useful representation for data analysis and machine learning task. **(5 Points)**
3. Count the total number of records per `fk_id` and compute the percentage of total data that each sensor contributed. **(5 Points)**
4. Replace **NULL** with `np.nan` (not a number from Numpy). Count how many valid lanes were used per entry and add a new column, `num_lanes_used`. **(3 Points)**
5. Identify potential sensor issues or outliers. Flag rows where `speed_std_dev > 10`, and rows where `flow == 0`, and then add a Boolean `is_anomaly` column. **(5 Points)**
6. Plot speed as a function of time using line plot. **(5 Points)**

8 Polars on EV Car Sales Data (10 Points)

Download EV car sales data from https://github.com/rahulbhadani/Datasets/blob/master/IEA-EV-dataEVsalesHistoricalCars.csv.

Use **polars** Python package for following data analysis:

1. Create two time series plots (one for BEV, one for PHEV) showing total EV sales over time globally (excluding *World*) for the top 3 countries with the highest overall EV sales. **(2 Points)**
2. For each country, calculate the year-over-year percentage growth in BEV sales. Which country had the highest growth in any year? Display the result as a sorted DataFrame. **(2 Points)**

3. Compute the average EV stock (parameter = "EV stock") between 2015 and 2020 for each country. Display the top 5 countries with the highest average EV stock using a horizontal bar chart. **(3 Points)**

4. For each year, compute the proportion of BEV vs. PHEV in total EV sales across all countries. Visualize the result using a stacked bar chart.**(3 Points)**

Through the next few questions, you will make a contribution to your Python package.

9 Cryptographically Secure Distribution (40 Points)

We will learn to generate random number drawn from a given distribution that is cryptographically secure. Consider the code snippets below:

```python
import secrets

def uniform(a: float = 0.0, b: float = 1.0) -> float:
    """Cryptographically secure uniform sample."""
    # 53 random bits gives 53-bit precision double
    u = secrets.randbits(53) / (1 << 53)   # in [0, 1)
    return a + (b - a) * u
```

Here, we use **secrets** module from Python that is designed for cryptographic security (e.g., generating keys, tokens), ensuring the randomness is unpredictable and suitable for security-sensitive applications. Floating-point numbers in Python (IEEE 754 double-precision) use 53 bits for precision. Generating 53 random bits ensures the output leverages the full precision of the float mantissa, avoiding bias or loss of granularity. Dividing the random 53-bit integer by 2^{53} (i.e., $1 << 53$) maps the value to the interval $[0, 1)$.

1. Your job is create a subpackage called `distributions` in your package. Inside your subpackage, create a module called `cvdistributions.py`. Add the above function to your module and demonstrate its use in your Colab notebook by generating 1000 random samples that are uniformly distributed. Create a **histogram plot** to demonstrate that 100000 random samples are indeed uniformly distributed. **(10 Points)**

   Note that samples generated this way are independent samples.

2. **Inverse Transform Sampling** method can be used to generate random samples from any distribution.

Inverse transform sampling requires us to know CDF of the desired probability distribution. A nice tutorial is provided by Prof. Ben Lambert at https://www.youtube.com/watch?v=rnBbYsysPaU.

Inverse Transform Sampling method says that if we know the CDF $F(x)$ of any distribution, we can generate random samples from its distribution. $F : \mathbb{R}to[0,1]$ is non-negative, non-decreasing, continuos from the right and has left hand limits.

We can write the inverse of F, $F^{-1} : [0,1] \rightarrow \mathbb{R}$ as

$$F^{-1}(y) = \min\{x : F(x) \geq y\}, y \in [0,1] \tag{16}$$

In this case, we can define the random variable corresponding to F as $X = F^{-1}(U)$ where U has the continuous uniform distribution over the interval $(0,1)$. Then X is distributed as F, that is, $P(X \leq x) = F(x), x \in \mathbb{R}$.

As an example, CDF of exponential distribution is $F(x) = 1 - e^{-\lambda}x, x \geq 0$ with $\lambda > 0$. Writing $y = 1 - e^{-\lambda x}$, where $y \in (0,1)$ we can rearrange it as $x = -\frac{1}{\lambda}\ln(1-y)$. This would be mean $X = -\frac{1}{\lambda}\ln(1-U)$. But if $U \sim Unif(0,1)$, then it means $1 - U \sim Unif(0,1)$, then we could also $1 - U$ by U as well.

Thus, we can get an exponentially distributed random sample in two steps:

(a) Generate a sample y from $U(0,1)$.

(b) Set $x = -\frac{1}{\lambda}\ln(y)$.

Repeat for all n samples you want to generate.

Create a function called **exponentialdist** in your `cvdistributions.py` module that takes λ as an argument and returns a rnadom sample that is exponentially distributed.

Plot a histogram of 100000 randomly distributed using exponential distribution in your Colab notebook.

(10 Points)

3. **Only for CPE 586 Students:**

   How would your algorithm modify if you need to generate random samples from a discrete distribution using inverse transform sampling technique? Your explanation should include necessary mathematical formulation as well.

   First write down the CDF of Poisson distribution and then write a function **poissiondist** takes a parameter λ in your `cvdistributions.py` module that generates Poisson distributed random samples.

   Plot a histogram of 100000 randomly distributed using Poisson distribution in your Colab notebook.

(20 Points)